

MVCC和临键锁的区别 以及MVCC无法解决幻读演示

mvcc无法解决幻读：

这是一个非常好的问题！**MVCC（多版本并发控制）** 和 **临键锁（Next-Key Lock）** 在 MySQL 的 **可重复读（Repeatable Read, RR）隔离级别** 下都用于解决并发问题，但它们的作用范围和能力有所不同。虽然 MVCC 可以在一定程度上解决幻读问题，但它并不能完全替代临键锁。以下是详细解释：

1. MVCC 如何解决幻读？

MVCC 通过为每个事务生成一个一致性视图（Consistent Read View），确保事务在读取数据时看到的是事务开始时的快照。对于普通的 `SELECT` 查询，MVCC 可以避免幻读，因为：

- 事务在读取数据时，只能看到事务开始前已经提交的数据。
- 其他事务在事务开始后插入的新数据（幻读）不会出现在当前事务的快照中。

示例：

```
-- 事务 A
START TRANSACTION;
SELECT * FROM users WHERE age > 20; -- 第一次查询，返回空集
-- 事务 B 插入 age = 25 的记录
SELECT * FROM users WHERE age > 20; -- 第二次查询，仍然返回空集（MVCC 避免幻读）
COMMIT;
```

在这个例子中，MVCC 确保了事务 A 在两次查询中看到的数据是一致的，避免了幻读。

2. MVCC 的局限性

https://blog.csdn.net/2301_79437276/article/details/139174746

MVCC+Next-key-Lock 防止幻读

InnoDB存储引擎在 RR 级别下通过 MVCC和 Next-key Lock 来解决幻读问题：两种解决方案尽可能避免幻读（超过间隙锁范围的控制不了）：

1、执行普通 select，此时会以 MVCC 快照读的方式读取数据

在快照读的情况下，RR 隔离级别只会在事务开启后的第一次查询生成 Read View，并使用至事务提交。所以在生成 Read View 之后其它事务所做的更新、插入记录版本对当前事务并不可见，实现了可重复读和防止快照读下（如果是update这些当前读就可以读到别人提交的。。）的“幻读”【借助第一次快照读时，只生成一次 readview】

2、执行 select...for update/lock in share mode、insert、update、delete 等当前读（MySQL 里除了普通查询是快照读，其他都是当前读，比如 update、insert、delete）-----【尽量在开启事务之后，马上执行 select ... for update 这类当前读的语句，因为它会对记录加 next-key lock，从而避免其他事务插入一条新记录。】-----如果用的普通select不会加next-key-lock所以会出现幻读。

在当前读下，读取的都是最新的数据，如果其它事务有插入新的记录，并且刚好在当前事务查询范围内，就会产生幻读！InnoDB 使用 来防止这种情况。当执行当前读时，会锁定读取到的记录的同时，锁定它们的间隙，防止其它事务在查询范围内插入数据。只要我不让你插入，就不会发生幻读。【next-key lock 是间隙锁+记录锁的组合】前提事务开启之后就执行当前读

InnoDB 引擎为了解决「可重复读」隔离级别使用「当前读」而造成的幻读问题，就引出了间隙锁。

比如：

事务 B 在执行插入语句的时候，判断到插入的位置被事务 A 加了 next-key lock，于是事务 B 会生成一个插入意向锁，同时进入等待状态，直到事务 A 提交了事务。这就避免了由于事务 B 插入新记录而导致事务 A 发生幻读的现象。

尽管 MVCC 可以避免普通的 `SELECT` 查询出现幻读，但它有以下局限性：

• 无法防止写操作导致的幻读：

- 如果事务 A 执行了写操作（如 `UPDATE` 或 `DELETE`），MVCC 的快照机制无法阻止其他事务插入新数据。
- 例如：

```
-- 事务 A
START TRANSACTION;
SELECT * FROM users WHERE age > 20; -- 第一次查询，返回空集
-- 事务 B 插入 age = 25 的记录
UPDATE users SET name = 'Alice' WHERE age > 20; -- 事务 A 的写操作
SELECT * FROM users WHERE age > 20; -- 第二次查询，返回 age = 25 的记录（幻读）
COMMIT;
```

在这个例子中，事务 A 的 `UPDATE` 操作会看到事务 B 插入的新数据，导致幻读。

（对于delete、update的不会，因为他们是当前读，不经过mvcc，所以才会有RR级别还会有幻读的问题，所以需要begin;之后马上执行当前读来锁住数据，不让其他的插入导致幻读。）

• 无法防止范围查询的幻读：

- 如果事务 A 执行了范围查询并加锁（如 `SELECT ... FOR UPDATE`），MVCC 无法阻止其他事务在范围内插入新数据。
- 例如：

```
-- 事务 A
START TRANSACTION;
SELECT * FROM users WHERE age > 20 FOR UPDATE; -- 锁定范围
-- 事务 B 插入 age = 25 的记录
SELECT * FROM users WHERE age > 20; -- 返回 age = 25 的记录（幻读）
COMMIT;
```

在这个例子中，事务 A 的 `SELECT ... FOR UPDATE` 操作无法阻止事务 B 插入新数据，导致幻读。

3. 临键锁的作用

临键锁（Next-Key Lock）是记录锁（Record Lock）和间隙锁（Gap Lock）的组合，用于锁定索引记录和记录之前的间隙。它可以：

- 防止其他事务在锁定范围内插入新数据，从而避免幻读。
- 防止其他事务修改锁定范围内的数据，从而避免不可重复读。

示例：

```
-- 事务 A
START TRANSACTION;
SELECT * FROM users WHERE age > 20 FOR UPDATE; -- 锁定范围 (20, +∞)
-- 事务 B 尝试插入 age = 25 的记录，会被阻塞
COMMIT;
```

在这个例子中，事务 A 的 `SELECT ... FOR UPDATE` 操作使用临键锁锁定了 `age > 20` 的范围，阻止了事务 B 插入新数据，从而避免了幻读。

4. MVCC 和临键锁的对比

特性	MVCC	临键锁 (Next-Key Lock)
作用	提供一致性视图，解决不可重复读问题。	锁定记录和间隙，解决幻读和不可重复读问题。
使用场景	普通 SELECT 查询（不加锁）。	加锁的 SELECT 查询和写操作（INSERT/UPDATE/DELETE）。
防止幻读	只能防止普通 SELECT 查询的幻读。	可以防止所有操作（包括写操作）的幻读。
阻塞性	非阻塞，读操作不会阻塞写操作。	阻塞，锁定范围内的操作会被阻塞。

5. 总结

- **MVCC:**
 - 用于普通 `SELECT` 查询，提供一致性视图，解决不可重复读问题。
 - 只能防止普通 `SELECT` 查询的幻读，无法防止写操作或加锁查询的幻读。
- **临键锁:**
 - 用于加锁的 `SELECT` 查询和写操作，解决幻读和不可重复读问题。
 - 可以防止所有操作（包括写操作）的幻读。

在 RR 隔离级别下，MySQL 会根据操作类型自动选择使用 MVCC 或临键锁：

- 如果是普通 `SELECT` 查询，使用 **MVCC**。
- 如果是加锁的 `SELECT` 查询或写操作，使用 **临键锁**。

因此，**MVCC 不能完全解决幻读问题，临键锁是必要的补充**。希望这个回答能帮助您更好地理解 MVCC 和临键锁的作用！如果还有其他问题，欢迎继续提问！

Mvcc和临键锁区别：

在 MySQL 的 **可重复读 (Repeatable Read, RR)** 隔离级别下，**MVCC (多版本并发控制)** 和 **临键锁 (Next-Key Lock)** 是两种核心机制，用于实现事务的隔离性和解决并发问题。它们的作用和使用场景有所不同，以下是详细说明：

1. MVCC (多版本并发控制)

作用：

- MVCC 通过为每个事务生成一个一致性视图 (Consistent Read View)，确保事务在读取数据时看到的是事务开始时的快照。
- MVCC 主要用于实现 **非阻塞读**，即读操作不会阻塞写操作，写操作也不会阻塞读操作。

使用场景：

- **普通 `SELECT` 查询：**
 - 当执行普通的 `SELECT` 查询（不加锁）时，MySQL 使用 MVCC 来提供一致性视图，确保事务看到的数据是事务开始时的快照。
 - 例如：

```
SELECT * FROM users WHERE id = 10;
```

这种查询不会加锁，而是通过 MVCC 读取快照数据。

- **解决不可重复读：**
 - MVCC 可以解决不可重复读问题（即同一事务中多次读取同一行数据，结果不一致）。
 - 例如：

```
-- 事务 A
START TRANSACTION;
SELECT * FROM users WHERE id = 10; -- 第一次读取
-- 事务 B 更新 id = 10 的行
SELECT * FROM users WHERE id = 10; -- 第二次读取，结果与第一次一致
COMMIT;
```

特点：

- **非阻塞**：读操作不会阻塞写操作，写操作也不会阻塞读操作。
- **快照读**：读取的是事务开始时的快照数据，而不是最新数据。

2. 临键锁 (Next-Key Lock)

作用：

- 临键锁是记录锁 (Record Lock) 和间隙锁 (Gap Lock) 的组合，用于锁定索引记录和记录之前的间隙。
- 临键锁主要用于防止 **幻读** 和 **不可重复读**，确保事务在锁定范围内的数据不会被其他事务修改或插入。

使用场景：

- **加锁的 SELECT 查询：**
 - 当执行加锁的 `SELECT` 查询 (如 `SELECT ... FOR UPDATE` 或 `SELECT ... LOCK IN SHARE MODE`) 时，MySQL 使用临键锁来锁定数据。
 - 例如：

```
SELECT * FROM users WHERE id = 10 FOR UPDATE;
```

这种查询会锁定 `id = 10` 的记录及其间隙，防止其他事务修改或插入数据。

- **防止幻读：**
 - 临键锁可以防止幻读问题 (即同一事务中多次执行相同的查询，结果集不一致)。
 - 例如：

```
-- 事务 A
START TRANSACTION;
SELECT * FROM users WHERE id > 10 AND id < 20 FOR UPDATE; -- 锁定范围 (10, 20]
-- 事务 B 尝试插入 id = 15 的记录，会被阻塞
COMMIT;
```

- **写操作 (INSERT/UPDATE/DELETE)：**
 - 当执行写操作时，MySQL 会自动使用临键锁来锁定相关记录和间隙。
 - 例如：

```
UPDATE users SET name = 'Alice' WHERE id = 10;
```

这种操作会锁定 `id = 10` 的记录及其间隙。

特点：

- **阻塞**：临键锁会阻塞其他事务的写操作（如插入、更新、删除）。
- **锁定范围**：不仅锁定记录，还锁定记录之前的间隙。

3. MVCC 和临键锁的对比

特性	MVCC	临键锁 (Next-Key Lock)
作用	提供一致性视图，解决不可重复读问题。	锁定记录和间隙，解决幻读和不可重复读问题。
使用场景	普通 <code>SELECT</code> 查询（不加锁）。	加锁的 <code>SELECT</code> 查询和写操作（ <code>INSERT/UPDATE/DELETE</code> ）。
阻塞性	非阻塞，读操作不会阻塞写操作。	阻塞，锁定范围内的操作会被阻塞。
锁定范围	无锁定，读取快照数据。	锁定记录和间隙。

4. 总结

- **MVCC**：
 - 用于普通 `SELECT` 查询，提供一致性视图，解决不可重复读问题。
 - 非阻塞，适合读多写少的场景。
- **临键锁**：
 - 用于加锁的 `SELECT` 查询和写操作，解决幻读和不可重复读问题。
 - 阻塞，适合需要严格数据一致性的场景。

在 RR 隔离级别下，MySQL 会根据操作类型自动选择使用 MVCC 或临键锁：

- 如果是普通 `SELECT` 查询，使用 **MVCC**。
- 如果是加锁的 `SELECT` 查询或写操作，使用 **临键锁**。

希望这个回答能帮助您更好地理解 MVCC 和临键锁的使用场景！如果还有其他问题，欢迎继续提问！